

JavaScript Variables

What is a Variable?

A **variable** is a named container used to store data in memory. It allows you to store, update, and reuse data in your program.

Example

```
let name = "Rokon";  
const age = 23;  
  
console.log(name);  
console.log(age);
```

Output

```
Rokon  
23
```

Types of JavaScript Variables

JavaScript has **3 types of variables**:

1. `var`
2. `let`
3. `const`

1. var

Explanation

- Old way of declaring variables.
- Value can be changed.
- Variable can be declared again.
- Function scoped.

Example

```
var city = "Dhaka";  
  
console.log(city);  
  
city = "Rajshahi";  
  
console.log(city);  
  
var city = "Khulna";  
  
console.log(city);
```

Output

```
Dhaka  
Rajshahi  
Khulna
```

2. let

Explanation

- Modern way of declaring variables.
- Value can be changed.
- Cannot be declared again in the same scope.
- Block scoped.

Example

```
let score = 50;  
  
console.log(score);  
  
score = 80;  
  
console.log(score);
```

Output

```
50  
80
```

Wrong Example

```
let score = 50;  
let score = 100; // Error
```

3. const

Explanation

- Modern way of declaring variables.
- Value cannot be changed after assignment.
- Cannot be declared again.
- Block scoped.

Example

```
const country = "Bangladesh";

console.log(country);
```

Output

```
Bangladesh
```

Wrong Example

```
const country = "Bangladesh";

country = "India"; // Error
```

Comparison

Feature	var	let	const
Reassign Value	Yes	Yes	No
Redeclare	Yes	No	No
Scope	Function	Block	Block
Recommended	No	Yes	Yes

Best Practice

- Use `const` by default.
- Use `let` if the value changes.
- Avoid `var` in modern JavaScript.

Summary

- `var` → Old method, can change and redeclare.
- `let` → Modern method, can change but cannot redeclare.
- `const` → Modern method, cannot change after assignment.

Variable Scope in JavaScript

Definition

Variable Scope is the area of a program where a variable can be accessed or used. It determines the visibility and lifetime of a variable.

Types of Variable Scope

1. Global Scope

A variable declared outside any function or block is called a **global variable**. It can be accessed from anywhere in the program.

Example

```
let name = "John";

function greet() {
  console.log(name);
}

greet();           // John
console.log(name); // John
```

2. Function Scope

A variable declared with `var` inside a function is only accessible within that function.

Example

```
function test() {
  var age = 20;
  console.log(age);
}

test();           // 20
console.log(age); // Error
```

3. Block Scope

A variable declared with `let` or `const` inside a block (`{ }`) can only be accessed within that block.

Example

```
if (true) {
  let x = 10;
  const y = 20;

  console.log(x); // 10
  console.log(y); // 20
}

console.log(x); // Error
console.log(y); // Error
```

var vs let vs const

Keyword	Scope	Can be Reassigned	Can be Redeclared
<code>var</code>	Function Scope	Yes	Yes
<code>let</code>	Block Scope	Yes	No
<code>const</code>	Block Scope	No	No

Importance of Variable Scope

- Prevents variable name conflicts.
- Improves code readability.
- Makes code easier to maintain.
- Protects variables from being accessed outside their intended area.
- Helps reduce bugs and unexpected behavior.

Summary

Scope	Description
Global Scope	Accessible from anywhere in the program.
Function Scope	Accessible only inside the function where it is declared.
Block Scope	Accessible only inside the block <code>{ }</code> where it is declared.

Key Points

- `var` → Function Scoped
- `let` → Block Scoped
- `const` → Block Scoped
- Global variables are accessible everywhere.
- Function variables cannot be accessed outside the function.
- Block-scoped variables cannot be accessed outside the block.

Interview Question

What is Variable Scope in JavaScript?

Answer:

Variable scope is the region of a program where a variable is accessible. JavaScript provides three main types of scope:

1. Global Scope
2. Function Scope
3. Block Scope

Variables declared with `let` and `const` are block-scoped, while variables declared with `var` are function-scoped.